11. ROS2 Parameter Service Case

1. Introduction to Parameters

Similar to global variables in C++ programming, parameters make it convenient to share certain data among multiple programs. A parameter is the global dictionary in the ROS robot system, which allows data to be shared among multiple running nodes.

In the ROS system, parameters exist in the form of a global dictionary. What is a dictionary? Just like a real dictionary, it consists of a name and a value, also called a key and a value, forming key-value pairs. Or we can also understand it as being like a parameter in programming: there is a parameter name, followed by an equals sign, and then the parameter value. When using it, you simply access this parameter name.

Parameters have very rich features. For example, if a certain node shares a parameter, all other nodes can access it. If a certain node modifies a parameter, other nodes also have a way to know immediately and obtain the latest value.

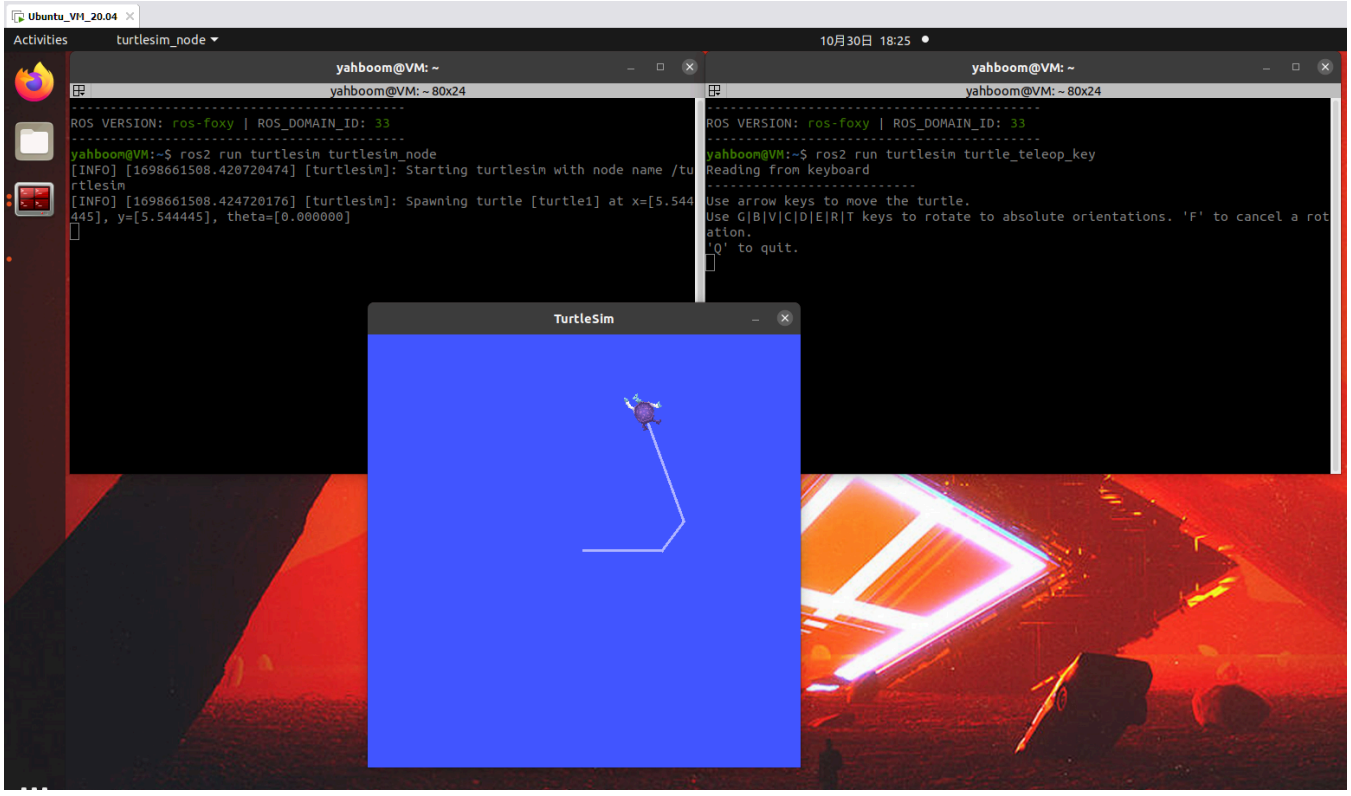
2. Parameters in the Little Turtle Example

In the little turtle example, the simulator also provides quite a few parameters. Let's use this example to get familiar with the meaning of parameters and the command line usage.

Since a GUI has to be displayed here, for the convenience of operation, the following cases are demonstrated in the virtual machine that comes with the tutorial.

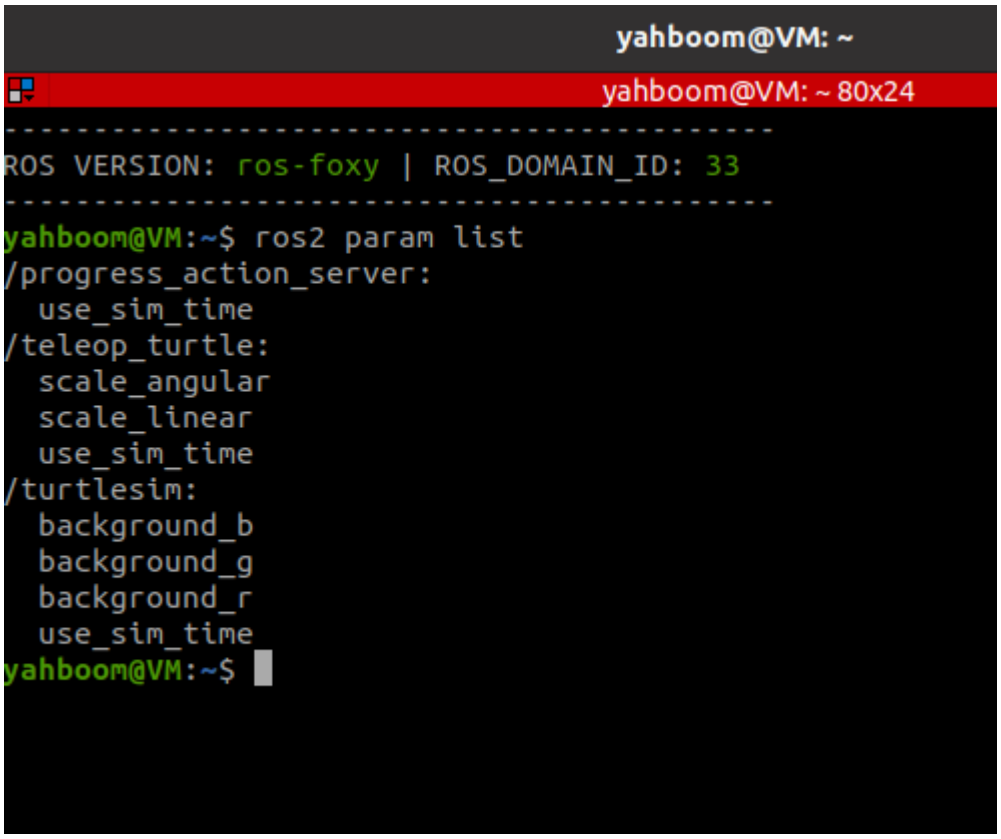
1. Launch two terminals and run the little turtle simulator and the keyboard control node respectively:

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```



2. Launch a terminal in the virtual machine and use the following command to view the parameter list

```
ros2 param list
```



3. Parameter query and modification

If you want to query or modify the value of a certain parameter, you can follow the param command with the get or set subcommand:

```
ros2 param describe turtlesim background_b   # View the description information of a
parameter
ros2 param get turtlesim background_b        # Query the value of a parameter
ros2 param set turtlesim background_b 10     # Modify the value of a parameter
```

4. Parameter file saving and loading

Querying/modifying parameters one by one is too troublesome. Instead, let's try parameter files. Parameter files in ROS use the yaml format. You can follow the param command with the dump subcommand to save all the parameters of a node into a file, or use the load command to load all the contents of a parameter file at once:

```
ros2 param dump turtlesim >> turtlesim.yaml  # Save the parameters of a node into a
parameter file
ros2 param load turtlesim turtlesim.yaml     # Load all the parameters in a file at once
```

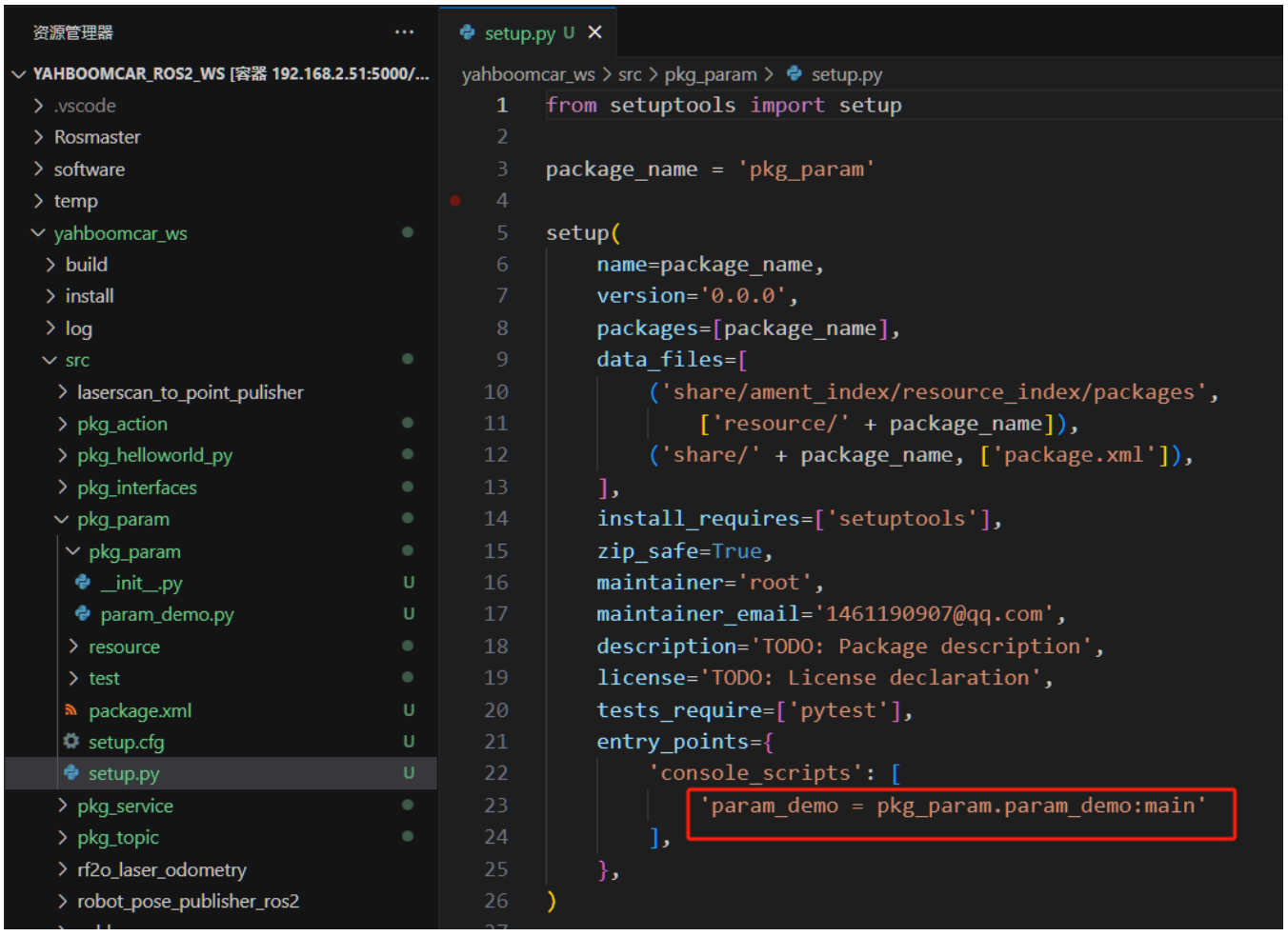
3. Parameter Case

3.1. Creating a New Function Package

- Create a new function package under the src directory of the workspace

```
ros2 pkg create pkg_param --build-type ament_python --dependencies rclpy --node-name
param_demo
```

After executing the above command, the pkg_param function package will be created, a param_demo node will also be created, and the related configuration files will have been configured.



3.2. Code Implementation

Next, edit [param_demo.py] to implement the publisher functionality and add the following code:

```
import rclpy                                    # ROS2 Python interface library
from rclpy.node import Node                     # ROS2 node class

class ParameterNode(Node):
    def __init__(self, name):
        super().__init__(name)                                  # ROS2 node parent class
initialization
        self.timer = self.create_timer(2, self.timer_callback)    # Create a timer (a period
in seconds, a callback function executed at a fixed interval)
```

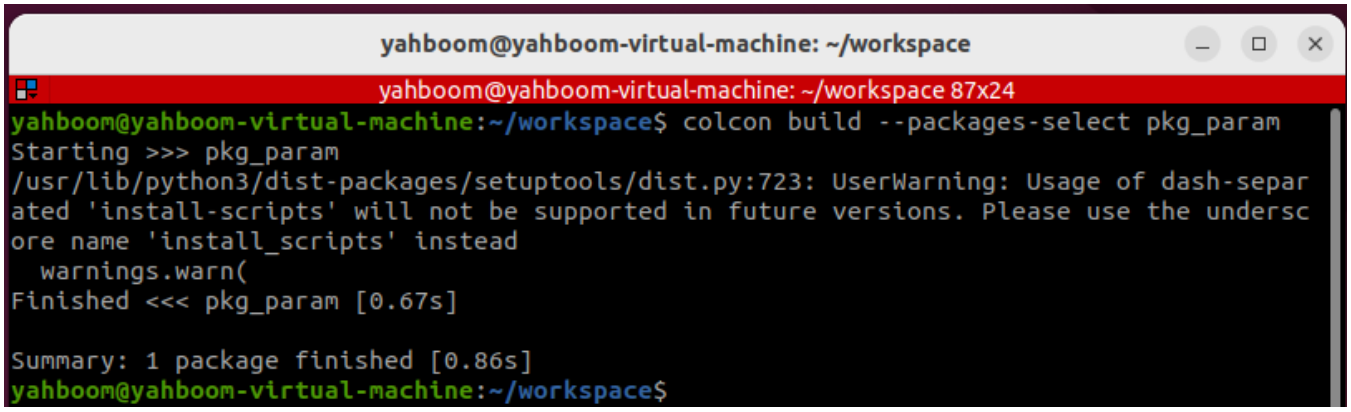
```
        self.declare_parameter('robot_name', 'muto')              # Create a parameter and
set its default value

    def timer_callback(self):                                      # Create the callback
function executed periodically by the timer
        robot_name_param =
self.get_parameter('robot_name').get_parameter_value().string_value   # Read the value of
the parameter from the ROS2 system
        self.get_logger().info('Hello %s!' % robot_name_param)     # Output log information
and print the parameter value that was read

def main(args=None):                                          # ROS2 node main entry main function
    rclpy.init(args=args)                                     # ROS2 Python interface initialization
    node = ParameterNode("param_declare")                     # Create a ROS2 node object and
initialize it
    rclpy.spin(node)                                          # Loop and wait for ROS2 to exit
    node.destroy_node()                                       # Destroy the node object
    rclpy.shutdown()                                          # Close the ROS2 Python interface
```

3.3. Compile the Function Package

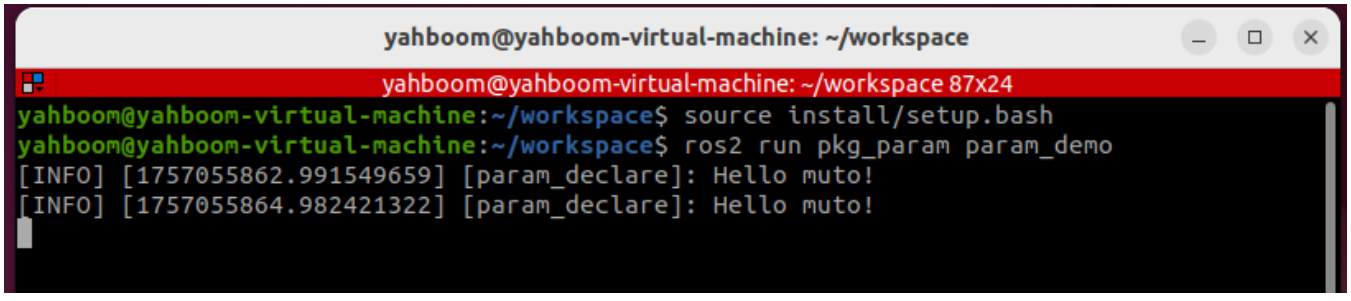
```
colcon build --packages-select pkg_param
```



3.4. Run the Program

- First refresh the environment variables, then run the node

```
ros2 run pkg_param param_demo
```



Open another terminal and set robot_name to robot:

```
ros2 param set param_declare robot_name robot
```

You can see the log information printed repeatedly in the terminal, where "muto" is a default parameter value we set, and the parameter name is "robot_name". After modifying this parameter through the command line, you can see that the terminal changes accordingly as well.

